

UAE E-Commerce Fraud Detection

Supervised Binary Classification

by Abel Chuchu

February 21, 2026

Introduction & Problem Statement

Objective:

The goal of my assignment is to develop a strong supervised machine learning model that can accurately predict whether an online transaction is fraudulent ($is_fraud = 1$) or legitimate ($is_fraud = 0$) based on a variety of transactional and behavioural features.

Dataset:

My dataset have 100,000 synthetic e-commerce transactions, designed to replicate real-world fraud patterns in the UAE online retail market. This dataset have over 40 features across several categories: transaction details, user profile, payment intelligence, device & network information, geographic data, velocity metrics, email intelligence, and risk flags.

Key Columns:

- transaction_id, user_id, timestamp_utc, amount_aed, currency
- payment_method, device_type, browser, merchant_category, items_count
- shipping_city, billing_city, shipping_billing_match, ip_address, ip_risk_score
- card_present, bin_country, card_age_days, card_country_match
- email_domain, user_prev_chargebacks, user_is_high_risk, user_account_age_days
- transactions_last_24h, transactions_last_1h, local_hour, odd_hour
- fraud_flag_ip, fraud_flag_mismatch, fraud_flag_velocity, fraud_flag_new_account, fraud_flag_prev_cb, fraud_flag_odd_hour
- **Target:**
is_fraud (0 = Legitimate, 1 = Fraudulent)

Dataset Link: [UAE E-Commerce Fraud on Kaggle](#)

Task:

Binary classification (fraud vs. non-fraud)

Target Variable:

My target variable is `is_fraud` – a binary indicator of fraudulent transactions.

Overview:

This assignment is about supervised learning, where the model learns from labelled data (transactions already marked as fraud or legitimate) to predict the outcome for new transactions. I will perform feature engineering (for example, log transform of "amount_aed", create a "high_velocity" flag) and carefully select a subset of the most informative features to build a strong classifier. In my dataset fraud is rare (only ~8% of transactions), so I must handle class imbalance by using techniques like stratified splitting, appropriate evaluation metrics (precision, recall, F1-score for the fraud class), and possibly class weighting. The final model will be compared across several algorithms (Logistic Regression, Decision Tree, Random Forest, SGD, Voting Ensemble) to select the best performer for this given business problem.

I chose fraud detection because I am taking a digital security course and a machine learning course and I wanted to connect these ideas. Plus I wanted to train a model that has real life use. And the dataset I found is realistic and related to my field. It has many significant real-world impact such as preventing financial losses, protecting customers, and improving trust in online platforms.

##Step 1: Import Libraries

```
"""
Step 1: Import Libraries
This part of the code is where I import all Python libraries I will be
using to visualize the data, preprocess it, and train the model.
"""

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.model_selection import train_test_split, cross_val_score,
GridSearchCV, StratifiedKFold
from sklearn.metrics import classification_report, confusion_matrix,
accuracy_score, precision_score, recall_score, f1_score, roc_auc_score
from sklearn.linear_model import LogisticRegression, SGDClassifier

from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
import pickle
import warnings
import kagglehub
import os
import time
```

```
warnings.filterwarnings('ignore')
print("All libraries imported successfully.")

All libraries imported successfully.
```

##Step 2: Load the Dataset

In this step, I downloaded the dataset using Kaggle API and load it into pandas Dataframe. I then used `df.head()` to show the first five rows to understand the columns and datatypes. I also used `df.info()` to see the data types and confirms there are no missing values.

```
"""
Step 2: Load the Dataset
This section loads the raw CSV data using Kaggle API and performs
initial checks.
"""
# Download dataset
path = kagglehub.dataset_download("atharvasoundankar/uae-e-commerce-
fraud")

# Dynamically find the CSV file in the downloaded folder
csv_files = [f for f in os.listdir(path) if f.endswith('.csv')]
csv_path = os.path.join(path, csv_files[0])

# Load the CSV
df = pd.read_csv(csv_path)

print("--- First 5 Rows ---")
display(df.head())
print("\n--- Dataset Info ---")
df.info()
print("\n--- Target Variable (is_fraud) Distribution ---")
print(df['is_fraud'].value_counts(normalize=True) * 100)

Warning: Looks like you're using an outdated `kagglehub` version
(installed: 0.3.13), please consider upgrading to the latest version
(1.0.0).
Downloading from
https://www.kaggle.com/api/v1/datasets/download/atharvasoundankar/uae-
e-commerce-fraud?dataset_version_number=1...

100%|██████████| 4.40M/4.40M [00:00<00:00, 34.0MB/s]

Extracting files...

--- First 5 Rows ---

{"type": "dataframe"}
```

--- Dataset Info ---

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 100000 entries, 0 to 99999

Data columns (total 36 columns):

#	Column	Non-Null Count	Dtype
0	transaction_id	100000 non-null	int64
1	user_id	100000 non-null	int64
2	timestamp_utc	100000 non-null	object
3	amount_aed	100000 non-null	float64
4	currency	100000 non-null	object
5	payment_method	100000 non-null	object
6	device_type	100000 non-null	object
7	browser	100000 non-null	object
8	merchant_category	100000 non-null	object
9	items_count	100000 non-null	int64
10	avg_item_price	100000 non-null	float64
11	shipping_city	100000 non-null	object
12	billing_city	100000 non-null	object
13	shipping_billing_match	100000 non-null	int64
14	ip_address	100000 non-null	object
15	ip_risk_score	100000 non-null	float64
16	card_present	100000 non-null	int64
17	bin_country	100000 non-null	object
18	card_age_days	100000 non-null	int64
19	card_country_match	100000 non-null	int64
20	email_domain	100000 non-null	object
21	user_prev_chargebacks	100000 non-null	int64
22	user_is_high_risk	100000 non-null	int64
23	user_account_age_days	100000 non-null	int64
24	transactions_last_24h	100000 non-null	int64
25	transactions_last_1h	100000 non-null	int64
26	local_hour	100000 non-null	int64
27	odd_hour	100000 non-null	int64
28	is_fraud	100000 non-null	int64
29	fraud_flag_ip	100000 non-null	int64
30	fraud_flag_mismatch	100000 non-null	int64
31	fraud_flag_velocity	100000 non-null	int64
32	fraud_flag_new_account	100000 non-null	int64
33	fraud_flag_prev_cb	100000 non-null	int64
34	fraud_flag_odd_hour	100000 non-null	int64
35	data_source	100000 non-null	object

dtypes: float64(3), int64(21), object(12)

memory usage: 27.5+ MB

--- Target Variable (is_fraud) Distribution ---

is_fraud

0 91.789

```
1      8.211
Name: proportion, dtype: float64
```

As I expected there are no missing values because I am using a synthetic dataset. All columns have 100,000 records. In this dataset, only 8.2% of the transactions are fraudulent. This means there is a clear imbalance so accuracy is very misleading.

For example, If I trained a model that label every transaction non fraud, it will be 91.8% accurate. Hence, I will be using other evaluation metrics like f1 and also use class-weighted learning

Step 3: Exploratory Data Analysis (EDA)

In this part, I will be visualizing the data to see if there is any pattern, check distributions, and inform feature engineering. I will be showing a few because I have about 20 features and showing all and their relation would be bulky and unessential.

```
"""
Step 3: Exploratory Data Analysis (EDA)
In this chunk, I perform exploratory data analysis with only the
specified graphs: pie chart for fraud distribution, histograms for
ip_risk_score, user_account_age_days, and transactions_last_24h,
correlation matrix for a selected subset of numeric features (to keep
it small and focused, dropping unimportant ones like IDs/flags), and
countplots for payment_method, device_type (device distribution),
browser, shipping_city, email_domain, and bin_country. This helps
identify patterns and insights from the chosen features.
"""

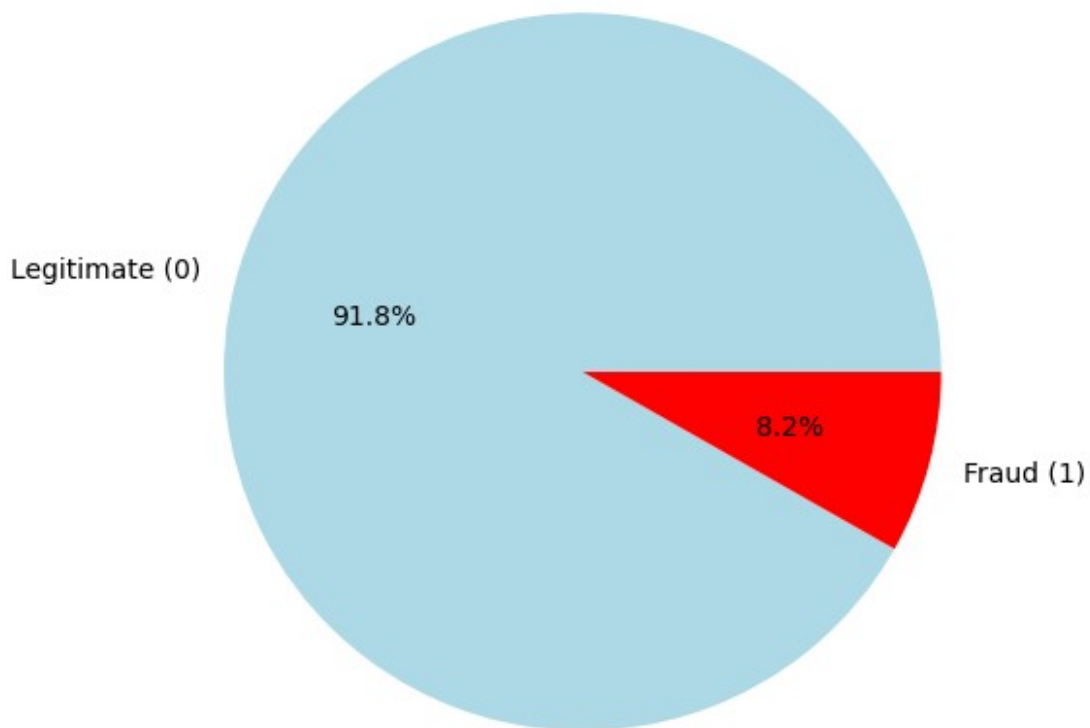
# Pie chart for fraud distribution
plt.figure(figsize=(6, 6))
df['is_fraud'].value_counts().plot.pie(autopct='%1.1f%%',
labels=['Legitimate (0)', 'Fraud (1)'], colors=['lightblue', 'red'])
plt.title('Distribution of Fraud Classes')
plt.ylabel('')
plt.show()

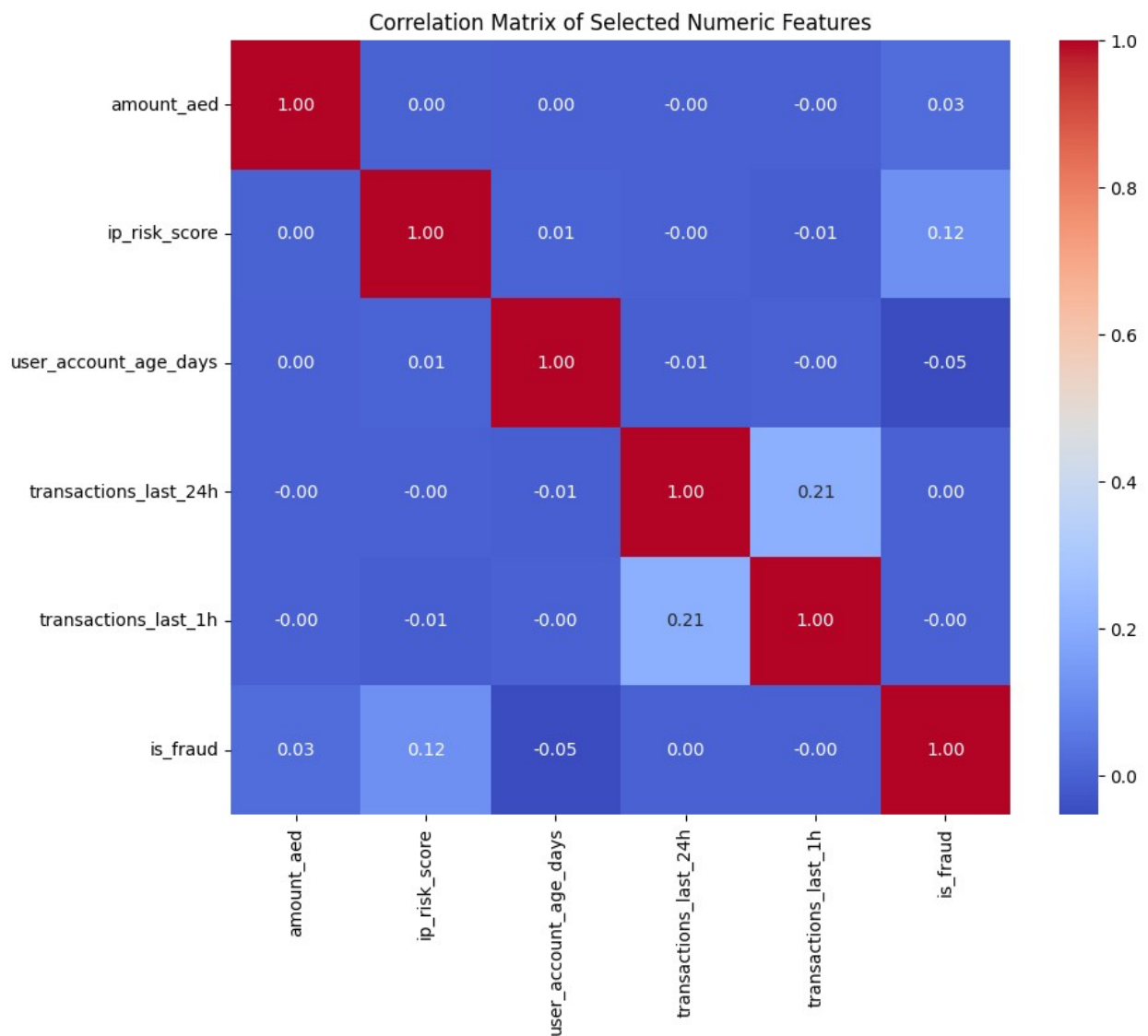
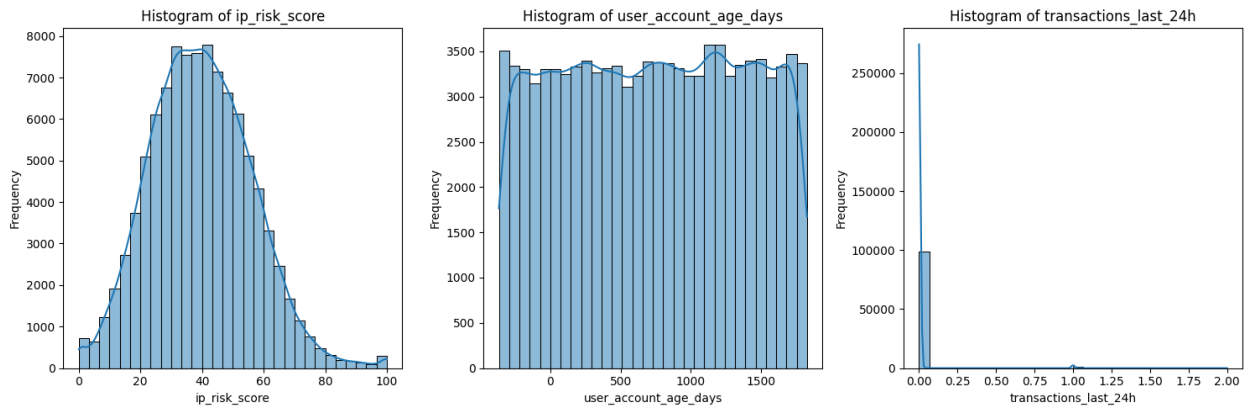
# Histograms for specified numeric features
numeric_to_plot = ['ip_risk_score', 'user_account_age_days',
'transactions_last_24h']
plt.figure(figsize=(15, 5))
for i, col in enumerate(numeric_to_plot, 1):
    plt.subplot(1, 3, i)
    sns.histplot(df[col], kde=True, bins=30)
    plt.title(f'Histogram of {col}')
    plt.xlabel(col)
    plt.ylabel('Frequency')
plt.tight_layout()
plt.show()

# Correlation matrix for a selected subset of numeric features
```

```
(dropped unimportant like IDs/flags to make it smaller; includes key
ones + is_fraud)
selected_numeric_for_corr = ['amount_aed', 'ip_risk_score',
                             'user_account_age_days', 'transactions_last_24h',
                             'transactions_last_1h', 'is_fraud']
plt.figure(figsize=(10, 8))
sns.heatmap(df[selected_numeric_for_corr].corr(), annot=True,
            cmap='coolwarm', fmt='.2f')
plt.title('Correlation Matrix of Selected Numeric Features')
plt.show()
```

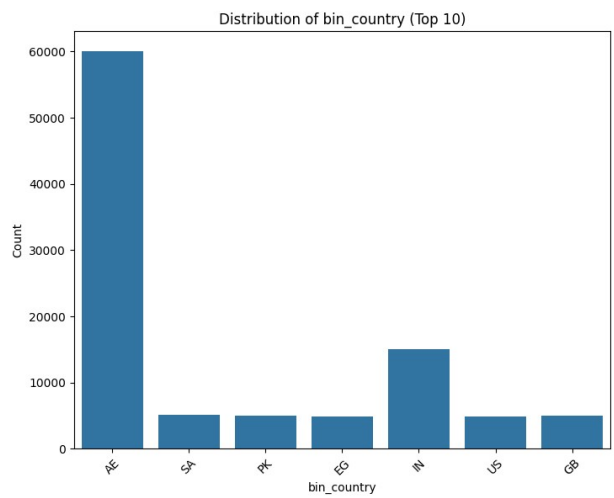
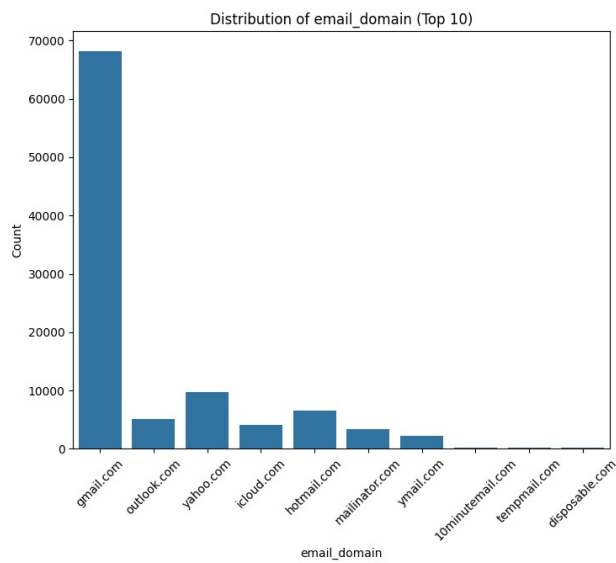
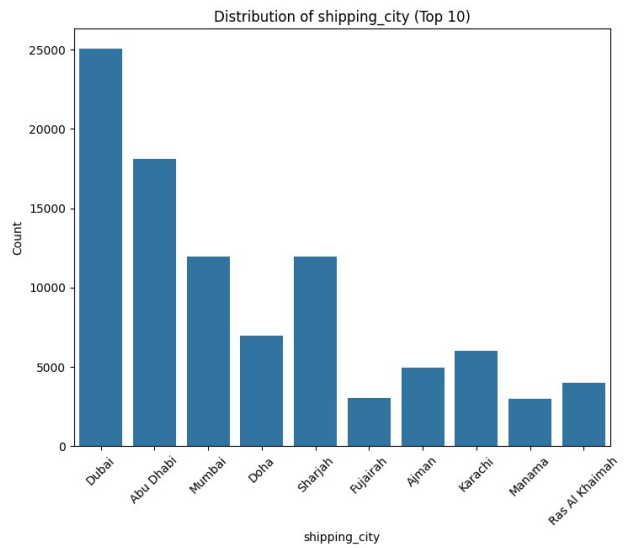
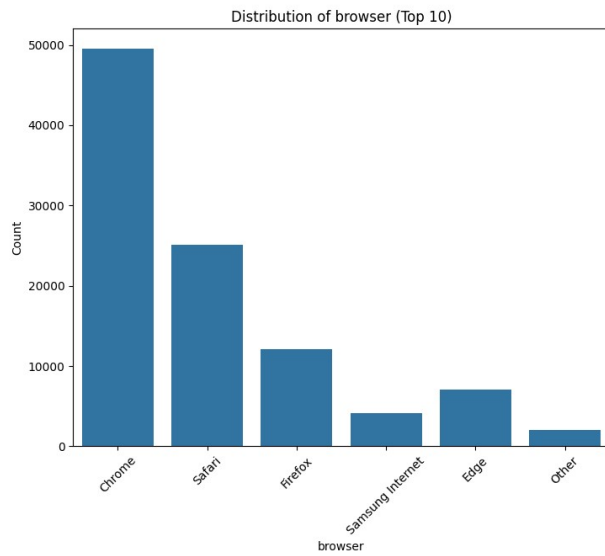
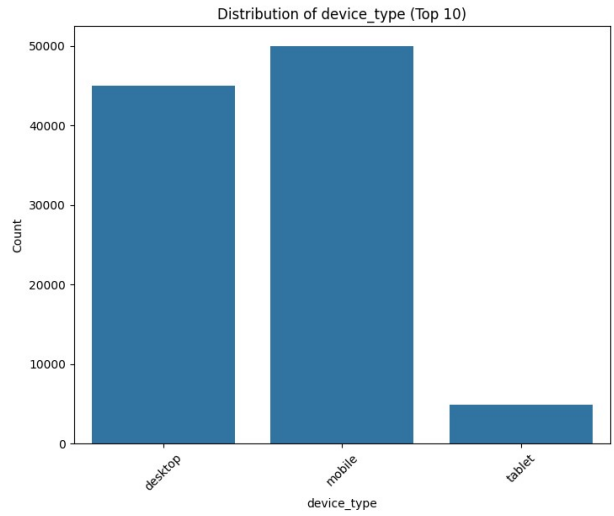
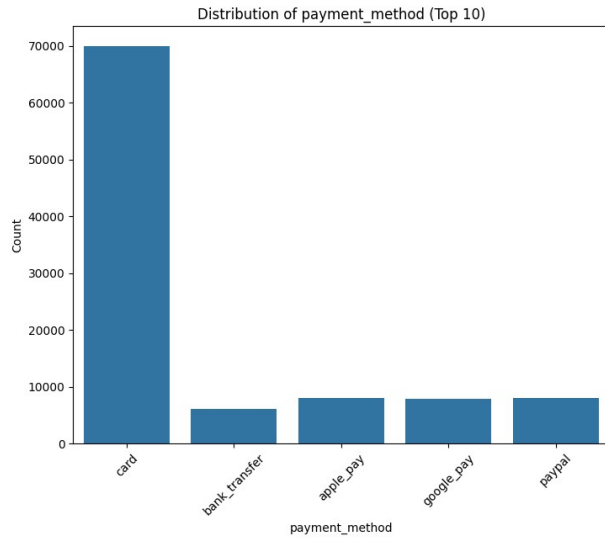
Distribution of Fraud Classes





```
# Countplots for specified categorical features (top 10 for those with
many categories)
categorical_to_plot = ['payment_method', 'device_type', 'browser',
```

```
'shipping_city', 'email_domain', 'bin_country']
plt.figure(figsize=(15, 20))
for i, col in enumerate(categorical_to_plot, 1):
    plt.subplot(3, 2, i)
    top_categories = df[col].value_counts().head(10).index
    sns.countplot(data=df[df[col].isin(top_categories)], x=col)
    plt.title(f'Distribution of {col} (Top 10)')
    plt.xlabel(col)
    plt.ylabel('Count')
    plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

I will be explaining each graph in the report but here is an overview of the result.

3.1 Fraud Distribution (Pie Chart)

The pie chart confirms the 91.8% / 8.2% split. This imbalance is my main challenge because I need to be careful of misinterpretation. I noted that a high overall accuracy is easy to achieve, but meaningless for fraud detection.

3.2 Histograms of Key Numeric Features

Histograms of `ip_risk_score`, `user_account_age_days`, and `transactions_last_24h` show:

- `ip_risk_score` is spread widely as some transactions have very high risk. I keep them as they may still carry signal.
- `transactions_last_24h` is heavily zero-inflated, but a few transactions have high values which I assume are velocity attacks.

3.3 Correlation Matrix

I used only a subset of numeric features plus the target `is_fraud`. The correlations with fraud are weak (largest is ~0.11 for `ip_risk_score`).

I was expecting this because fraud patterns are often non-linear, so linear correlation understates their predictive power. The matrix reassures me that features are not highly correlated with each other (no multicollinearity), which is good for linear models.

3.4 Count Plots of Categorical Features

Categorical Count Plots – I examined the top 10 categories for:

- `payment_method` – dominated by 'card' and 'bank_transfer'.
- `device_type` – mainly 'desktop' and 'mobile'.
- `browser` – Chrome, Safari, etc.
- `shipping_city`, `email_domain`, `bin_country` – show a long tail of values.

Some of my categorical features (like `payment_method`, `device_type`, `browser`, etc.) contain many different categories, and some appear only a few times. If I used Label Encoding, each category would be turned into a number (For example, card = 1, apple_pay = 2, paypal = 3). But this creates a fake ranking because it makes the model think `paypal > apple_pay > card`, even though these categories have no natural order.

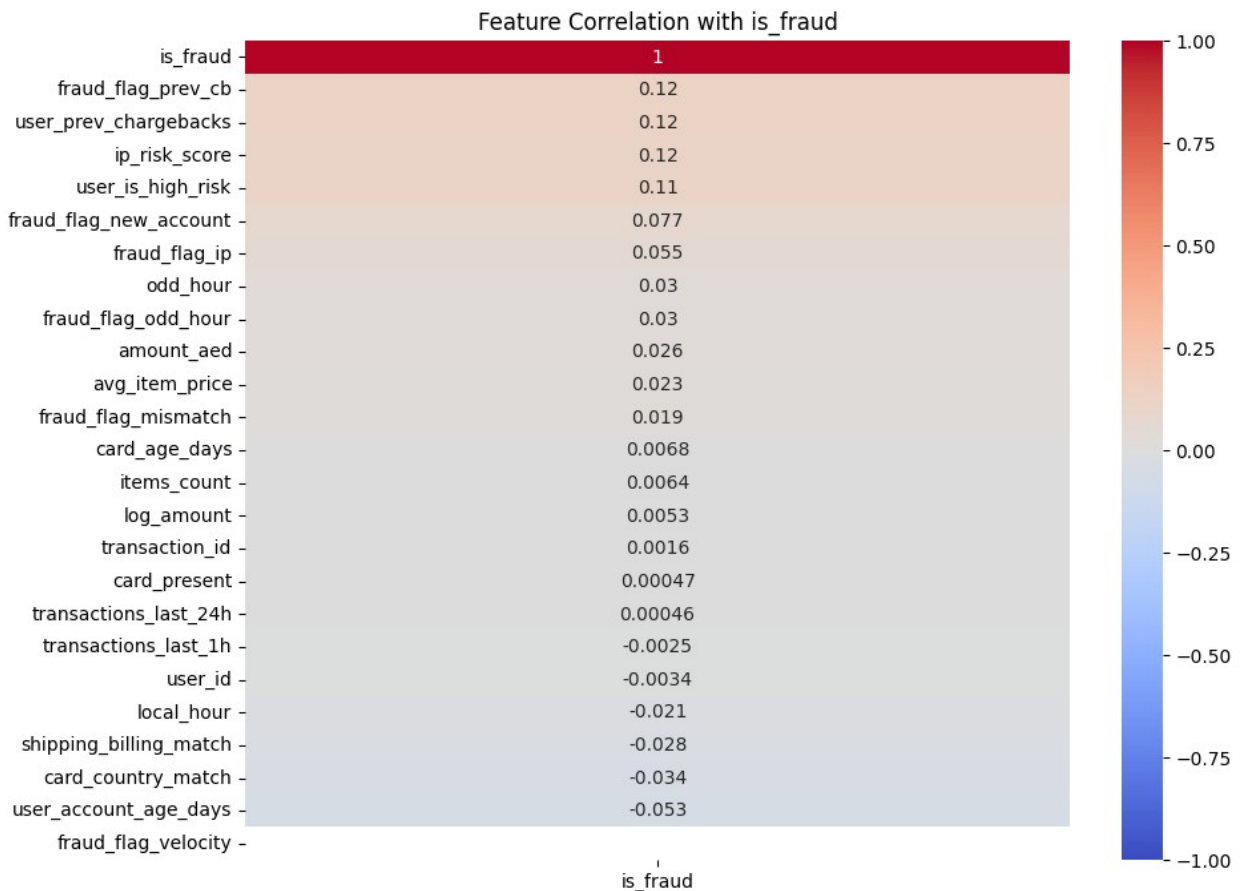
To avoid this problem, I used One-Hot Encoding, which creates a separate binary column for each category (For example, `is_card`, `is_paypal`, `is_apple_pay`). This treats all categories equally and prevents the model from assuming any artificial hierarchy. It is the correct method for unordered categorical variables.

```
# Correlation Heatmap (Numeric features only)
plt.figure(figsize=(10, 8))
numeric_cols = df.select_dtypes(include=[np.number]).columns
correlation_matrix = df[numeric_cols].corr()
# I am only showing correlation with the target to keep it focused
```

```

sns.heatmap(correlation_matrix[['is_fraud']].sort_values(by='is_fraud'
, ascending=False),
            annot=True, cmap='coolwarm', vmin=-1, vmax=1)
plt.title('Feature Correlation with is_fraud')
plt.show()

```



By looking at this table, I **deliberately avoided features such as fraud_flag_new_account, fraud_flag_ip, and similar fraud_flag_ip** eventhough they have high correlation because those features are likely derived from predefined fraud rules. Using them could introduce data leakage of the target variable . Including them could introduce data leakage, where the model indirectly learns the answer from features that already encode fraud logic. Instead, I selected original predictors such as ip_risk_score, user_account_age_days, transactions_last_1h, user_prev_chargebacks, and card_country_match, which allow the model to discover patterns on its own. I also performed feature engineering by transforming amount_aed into log_amount to reduce skewness and limit the influence of extreme outliers.

Step 4: Data Cleaning, Pre-processing, and Feature Engineering

In this step, I cleaned the dataset and selected the most important features for modeling. I checked for missing values, remived unnecessary columns, selected only 8 meaningful features

and separated numerical and caregorical features. I also created a new feature called `log_amount` for feature engineering. **In this step I defined X to be all selected features whereas y is is_fraud. Hence all legitimate are 0 and fraud is marked as 1.**

```
"""
Step 4: Data Cleaning, Pre-processing, and Feature Engineering
First, I check for missing values to understand the dataset's quality.
Then, I select exactly 8 features to meet the assignment requirements,
engineer a log_amount feature to handle skewness, and cap extreme
outliers.
"""

import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# --- 4.0: Checking for Missing Values ---
missing_values = df.isnull().sum()
missing_values = missing_values[missing_values >
0].sort_values(ascending=False)

print("--- Missing Values Check ---")
if missing_values.empty:
    print("Result: There are NO missing values in the dataset!\n")
else:
    print(missing_values)
    plt.figure(figsize=(8, 4))
    sns.barplot(x=missing_values.values, y=missing_values.index,
palette='Reds_r')
    plt.title('Count of Missing Values per Column')
    plt.xlabel('Number of Missing Rows')
    plt.ylabel('Features')
    plt.show()

# --- 4.1: Feature Engineering & Selection ---
# Create log_amount to handle skewed currency values
df['log_amount'] = np.log1p(df['amount_aed'])

# 1. Select EXACTLY 6 features + the target
# 4 Numeric: log_amount, ip_risk_score, user_account_age_days,
transactions_last_24h
# 2 Categorical: payment_method, device_type
features_to_keep = [
    'log_amount',
    'ip_risk_score',
    'user_account_age_days',
    'transactions_last_1h',
    'user_prev_chargebacks',
    'card_country_match',
    'payment_method',
    'device_type',
```

```

    'is_fraud'
]

# 2. Filter the dataframe to only these columns and drop any missing
values just in case
df_cleaned = df[features_to_keep].dropna()

# Outlier Handling: Cap the 'log_amount' at the 99th percentile
df_cleaned['log_amount'] = np.clip(df_cleaned['log_amount'], None,
df_cleaned['log_amount'].quantile(0.99))

print(f"Dataset shape after cleaning and strictly selecting 6
features: {df_cleaned.shape}")
display(df_cleaned.head())

# 4. Define X and y for modeling
X = df_cleaned.drop(columns=['is_fraud'])
y = df_cleaned['is_fraud']

--- Missing Values Check ---
Result: There are NO missing values in the dataset!

Dataset shape after cleaning and strictly selecting 6 features:
(100000, 9)

{"summary": "{\n  \"name\": \"y = df_cleaned['is_fraud']\", \n
  \"rows\": 5, \n  \"fields\": [\n    {\n      \"column\":
  \"log_amount\", \n      \"properties\": {\n        \"dtype\":
  \"number\", \n        \"std\": 0.8296949932229948, \n        \"min\":
  4.26619481914876, \n        \"max\": 6.273631438871653, \n
  \"num_unique_values\": 5, \n        \"samples\": [\n
  4.26619481914876, \n        6.273631438871653, \n
  4.764308407326388\n      ], \n        \"semantic_type\": \"\", \n
  \"description\": \"\" \n    }, \n    {\n      \"column\":
  \"ip_risk_score\", \n      \"properties\": {\n        \"dtype\":
  \"number\", \n        \"std\": 6.441273166075166, \n        \"min\":
  18.5, \n        \"max\": 35.3, \n        \"num_unique_values\": 5, \n
  \"samples\": [\n        30.2, \n        18.5, \n        32.7\n      ], \n        \"semantic_type\": \"\", \n
  \"description\": \"\" \n    }, \n    {\n      \"column\":
  \"user_account_age_days\", \n      \"properties\": {\n        \"dtype\":
  \"number\", \n        \"std\": 783, \n        \"min\": -192, \n
  \"max\": 1744, \n        \"num_unique_values\": 5, \n        \"samples\": [\n
  -192, \n        -59, \n        1744\n      ], \n        \"semantic_type\": \"\", \n
  \"description\": \"\" \n    }, \n    {\n      \"column\":
  \"transactions_last_1h\", \n      \"properties\": {\n        \"dtype\":
  \"number\", \n        \"std\": 0, \n        \"min\": 0, \n
  \"max\": 0, \n        \"num_unique_values\": 1, \n        \"samples\":
  [\n        0\n      ], \n        \"semantic_type\": \"\", \n

```


matrix, there are no multicollinearity as the features are not strongly correlated with each other. Hence, PCA is unnecessary for redundancy removal.

Just to show my understanding, i will use PCA for visualization. I am using PCA only for visualization and not for feature designing or dimensionality reduction. By squishing the numeric features into 2 dimensions, I can plot them on an X/Y graph to see if Fraud and Legitimate transactions naturally form separate clusters. However I am not using it for training the model because

```
"""
PCA 2D Visualization
I am using Principal Component Analysis (PCA) strictly for
visualization purposes.
"""

from sklearn.decomposition import PCA

# Grab just the numeric features
X_numeric = df_cleaned[['log_amount', 'ip_risk_score',
                        'user_account_age_days', ]]

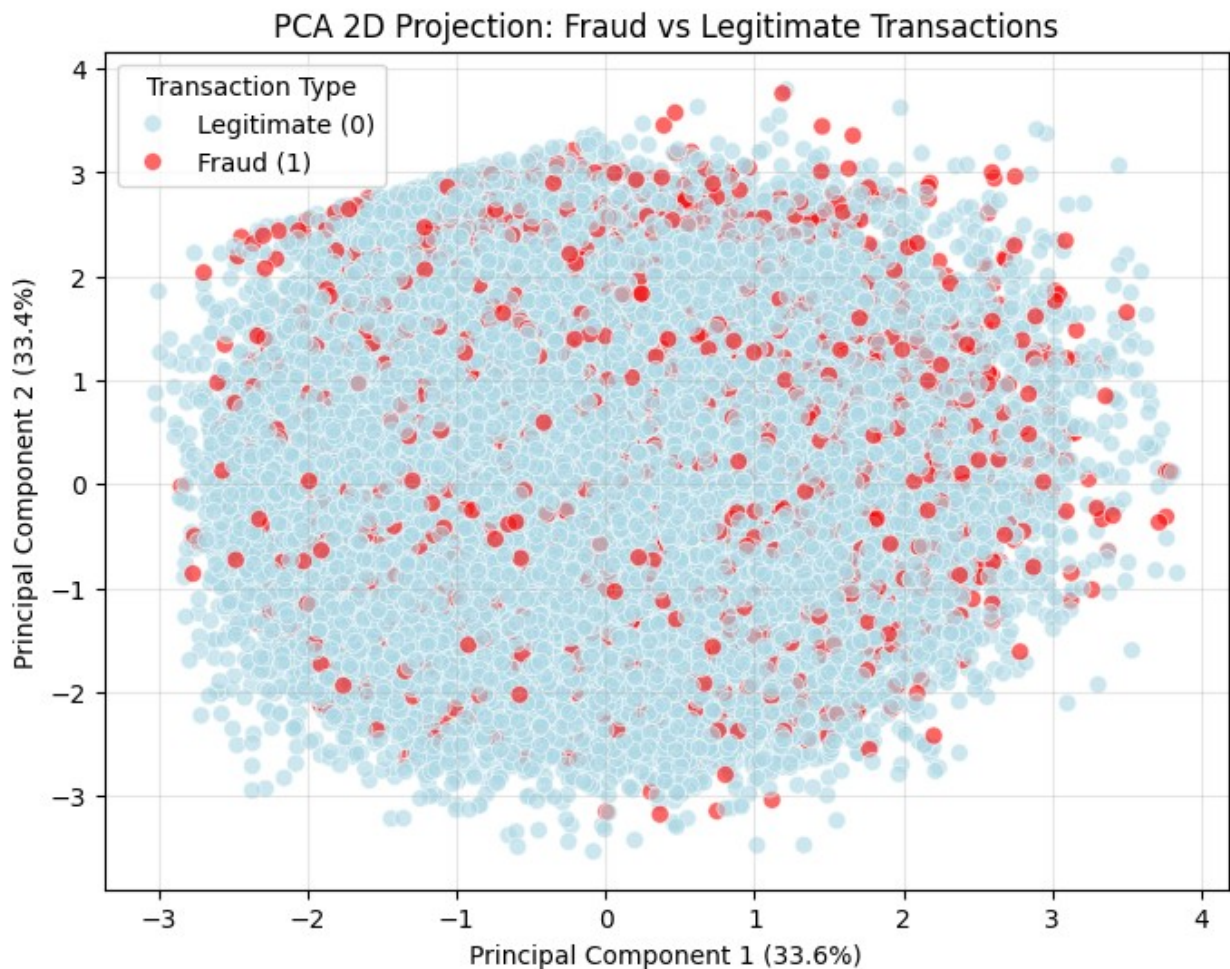
# Scale the data (PCA requires scaled data to work properly)
scaler_pca = StandardScaler()
X_scaled_pca = scaler_pca.fit_transform(X_numeric)

# Apply PCA to reduce to 2 dimensions
pca = PCA(n_components=2)
X_pca_2d = pca.fit_transform(X_scaled_pca)

# Plot the PCA
plt.figure(figsize=(8, 6))
sns.scatterplot(
    x=X_pca_2d[:, 0],
    y=X_pca_2d[:, 1],
    hue=df_cleaned['is_fraud'],
    palette=['lightblue', 'red'],
    alpha=0.6,
    s=50
)
plt.title('PCA 2D Projection: Fraud vs Legitimate Transactions')
plt.xlabel(f'Principal Component 1\n({pca.explained_variance_ratio_[0]*100:.1f}%)')
plt.ylabel(f'Principal Component 2\n({pca.explained_variance_ratio_[1]*100:.1f}%)')
legend = plt.legend(title='Transaction Type')
legend.get_texts()[0].set_text('Legitimate (0)')
legend.get_texts()[1].set_text('Fraud (1)')
plt.grid(alpha=0.3)
plt.show()
```



```
print(f"Total variance explained by these 2 components: {sum(pca.explained_variance_ratio_)*100:.1f}%")
```



Total variance explained by these 2 components: 67.0%

Here, I compressed my 4 numeric features down into 2 principal components so I can plot them on a 2D scatter plot and visually inspect if the fraud classes are separable. The scatter plot shows overlap, and this confirms the difficulty of the task. The two components explain about 67% of the variance, meaning some information is lost.

Step 5: Train/Test Split

In this step, I split the data into 80% training and 20% testing sets using stratified splitting to keep the fraud ratio (~8%) the same in both, avoiding bias.

I used stratified splitting because the dataset is highly imbalanced and if I randomly split the data without stratification, One might contain very few fraud cases. This makes the evaluation very unreliable.

As I explained earlier, If I predicted not-fraud for every transactions, I would still be 91.8%

accurate so I needed to carefully split the data and later focus only in F1 Binary instead of accuracy.

```
"""
Step 5: Train/Test Split
"""
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.20, random_state=42, stratify=y)
print(f"Training set size: {X_train.shape[0]} transactions")
print(f"Testing set size: {X_test.shape[0]} transactions")
```

```
Training set size: 80000 transactions
Testing set size: 20000 transactions
```

Step 6: Model Pipelines

In this step, I created pipelines for each model. Using pipeline is important because using chain preprocessing and model fitting, preventing data leakage, simplify cross-validation and hyperparameter tuning and also they make the code cleaner and less error-prone.

Preprocessing steps:

- For numeric features, I used `StandardScaler` which scales to zero mean and unit variance. This is necessary for distance-based models (Logistic Regression, SVM, KNN) and regularised models.
 - For categorical features, I used `OneHotEncoder` which converts categories into binary columns.
- In this assignment, I included linear regression, decision tree, random forest, SDGClassifier, and voting ensemble. Additionally I included SVC(RBF kernel) and K_Nearest Neighbors(KNN) to show why they are not effective or why I shouldn't use them.

```
"""
Step 6: Model Pipelines
Defining base models with strict regularizations to prevent
overfitting.
"""
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import VotingClassifier, RandomForestClassifier
from sklearn.linear_model import LogisticRegression, SGDClassifier
from sklearn.tree import DecisionTreeClassifier

# Strictly defining the 6 features
numeric_features = [
    'log_amount',
    'ip_risk_score',
    'user_account_age_days',
    'transactions_last_1h',
```

```

        'user_prev_chargedbacks',
        'card_country_match'
    ]

    categorical_features = ['payment_method', 'device_type']

    # Create the Preprocessor
    preprocessor = ColumnTransformer(
        transformers=[
            ('num', StandardScaler(), numeric_features),
            ('cat', OneHotEncoder(handle_unknown='ignore'),
             categorical_features)
        ]
    )

    # 1. BASE CLASSIFIERS
    clf_lr = LogisticRegression(C=0.1, max_iter=1000,
                                class_weight='balanced', random_state=42)
    clf_dt = DecisionTreeClassifier(max_depth=6, min_samples_split=20,
                                    class_weight='balanced', random_state=42)
    clf_rf = RandomForestClassifier(n_estimators=100, max_depth=6,
                                   class_weight='balanced', random_state=42)
    # Here is the SGD Classifier!
    clf_sgd = SGDClassifier(loss='log_loss', alpha=0.01, max_iter=1000,
                             class_weight='balanced', random_state=42)

    # 2. Voting Classifier (combining all 4)
    clf_voting = VotingClassifier(
        estimators=[('lr', clf_lr), ('dt', clf_dt), ('rf', clf_rf),
                    ('sgd', clf_sgd)],
        voting='soft'
    )

    # 3. Define the Model Pipelines
    pipe_lr = Pipeline([('preprocessor', preprocessor), ('clf', clf_lr)])
    pipe_dt = Pipeline([('preprocessor', preprocessor), ('clf', clf_dt)])
    pipe_rf = Pipeline([('preprocessor', preprocessor), ('clf', clf_rf)])
    # Here is where pipe_sgd is officially created!
    pipe_sgd = Pipeline([('preprocessor', preprocessor), ('clf',
                                                           clf_sgd)])
    pipe_voting = Pipeline([('preprocessor', preprocessor), ('clf',
                                                             clf_voting)])

    tuned_pipelines = {} # Initialize dictionary for Step 8
    print("All pipelines (including SGD) have been safely defined.")

    All pipelines (including SGD) have been safely defined.

```

Step 7: Model Selection and Base model training

In this step, I trained each base model using `fit()`. Base model means the models before I used hyperparameter tuning or simply training models before they are tuned. I also measured their performance and compared their metrics. The reason I selected the models is mainly because we learned about them in class and to show my understanding but additionally it is because of the following reasons:

1. **Logistic Regression** – is simple, fast, interpretable linear model. With `class_weight='balanced'`, it automatically adjusts for class imbalance by giving more weight to the minority class. It optimizes **Log Loss** (cross-entropy loss) which is its loss function.
2. **Decision Tree** – is a tree based model that minimizes Gini Impurity. I chose it because it captures complex patterns, does not require scaling, and it is easier to interpret but it is prone to overfitting.
3. **Random Forest** – an ensemble of decision trees, more robust and less overfitting than a single tree. Also supports `class_weight='balanced'`.
4. **SGDClassifier** – is a linear model trained using SGD and I used it because it is efficient for large dataset, is good for comparison with logistic regression and with `class_weight='balanced'` and shows gradient based optimization.
5. **Voting Ensemble** – combines the four models using soft voting (averaging probabilities). I used it to test whether combining models improve performance and to show my understanding of Ensemble method we learned in class. I used SVC but commented it out because it is very slow comparative to the other models. Its training time was too long and is very inefficient for large datasets. I mentioned to show my understanding. I also didn't use KNN because first I tried it and it didn't give me a much different result and it was overfitting a lot even after I tried tuning it again and again. Plus we didn't cover this in our course so I didn't include KNN and XGBOOST training models.

```
"""
Step 7.0: Setup Baseline Storage
I am creating an empty list to store the baseline results as we run
each model separately.
"""
import time
baseline_results = []
print("Baseline storage ready. Proceed to train individual models.")

Baseline storage ready. Proceed to train individual models.

"""
Step 7.1: Base Logistic Regression
"""
print("--- Training Logistic Regression (Base) ---")
pipe_lr.fit(X_train, y_train)

start_time = time.time()
y_pred_lr = pipe_lr.predict(X_test)
y_prob_lr = pipe_lr.predict_proba(X_test)[: , 1] # Extract
```

```

probabilities for Class 1 (Fraud)
test_time = time.time() - start_time

acc = accuracy_score(y_test, y_pred_lr)
f1 = f1_score(y_test, y_pred_lr, average='binary')
roc_auc = roc_auc_score(y_test, y_prob_lr) # Calculate ROC AUC

baseline_results.append({
    'Model': 'Logistic Regression',
    'Before_Accuracy': round(acc, 4),
    'Before_F1': round(f1, 4),
    'Before_ROC_AUC': round(roc_auc, 4), # Save ROC AUC
    'Before_Test_Time(s)': round(test_time, 5)
})
print(f"Done! Accuracy: {acc:.4f} | Binary F1: {f1:.4f} | ROC AUC: {roc_auc:.4f}")

```

```

--- Training Logistic Regression (Base) ---

```

```

Done! Accuracy: 0.6411 | Binary F1: 0.2152 | ROC AUC: 0.6788

```

```

"""

```

```

Step 7.2: Base SVC (RBF)

```

```

I am leaving this code here for demonstration. Training this base SVC
on 80,000 rows takes excessive compute time, and tuning it would take
hours.

```

```

"""

```

```

#print("--- Training SVC (Base) ---")

```

```

#pipe_svc.fit(X_train, y_train)

```

```

#start_time = time.time()

```

```

#y_pred_svc = pipe_svc.predict(X_test)

```

```

#test_time = time.time() - start_time

```

```

#acc = accuracy_score(y_test, y_pred_svc)

```

```

#f1 = f1_score(y_test, y_pred_svc, average='binary')

```

```

#baseline_results.append({'Model': 'SVC (RBF)', 'Before_Accuracy':
round(acc, 4), 'Before_F1': round(f1, 4), 'Before_Test_Time(s)':
round(test_time, 5)})

```

```

#print(f"Done! Accuracy: {acc:.4f} | F1: {f1:.4f}")

```

```

{"type": "string"}

```

```

"""

```

```

Step 7.2: Base Decision Tree (With ROC AUC)

```

```

"""

```

```

print("--- Training Decision Tree (Base) ---")

```

```

pipe_dt.fit(X_train, y_train)

```

```

start_time = time.time()

```

```

y_pred_dt = pipe_dt.predict(X_test)

```

```

y_prob_dt = pipe_dt.predict_proba(X_test)[: , 1]
test_time = time.time() - start_time

acc = accuracy_score(y_test, y_pred_dt)
f1 = f1_score(y_test, y_pred_dt, average='binary')
roc_auc = roc_auc_score(y_test, y_prob_dt)

baseline_results.append({
    'Model': 'Decision Tree',
    'Before_Accuracy': round(acc, 4),
    'Before_F1': round(f1, 4),
    'Before_ROC_AUC': round(roc_auc, 4),
    'Before_Test_Time(s)': round(test_time, 5)
})
print(f"Done! Accuracy: {acc:.4f} | Binary F1: {f1:.4f} | ROC AUC: {roc_auc:.4f}")

--- Training Decision Tree (Base) ---
Done! Accuracy: 0.5776 | Binary F1: 0.2108 | ROC AUC: 0.6739

"""
Step 7.3: Base Random Forest (With ROC AUC)
"""
print("--- Training Random Forest (Base) ---")
pipe_rf.fit(X_train, y_train)

start_time = time.time()
y_pred_rf = pipe_rf.predict(X_test)
y_prob_rf = pipe_rf.predict_proba(X_test)[: , 1]
test_time = time.time() - start_time

acc = accuracy_score(y_test, y_pred_rf)
f1 = f1_score(y_test, y_pred_rf, average='binary')
roc_auc = roc_auc_score(y_test, y_prob_rf)

baseline_results.append({
    'Model': 'Random Forest',
    'Before_Accuracy': round(acc, 4),
    'Before_F1': round(f1, 4),
    'Before_ROC_AUC': round(roc_auc, 4),
    'Before_Test_Time(s)': round(test_time, 5)
})
print(f"Done! Accuracy: {acc:.4f} | Binary F1: {f1:.4f} | ROC AUC: {roc_auc:.4f}")

--- Training Random Forest (Base) ---
Done! Accuracy: 0.5982 | Binary F1: 0.2166 | ROC AUC: 0.6801

"""
Step 7.4: Base SGD Classifier (With ROC AUC)
Because we used loss='log_loss', SGD supports probability predictions!

```

```

"""
print("--- Training SGD Classifier (Base) ---")
pipe_sgd.fit(X_train, y_train)

start_time = time.time()
y_pred_sgd = pipe_sgd.predict(X_test)
y_prob_sgd = pipe_sgd.predict_proba(X_test)[: , 1]
test_time = time.time() - start_time

acc = accuracy_score(y_test, y_pred_sgd)
f1 = f1_score(y_test, y_pred_sgd, average='binary')
roc_auc = roc_auc_score(y_test, y_prob_sgd)

baseline_results.append({
    'Model': 'SGD Classifier',
    'Before_Accuracy': round(acc, 4),
    'Before_F1': round(f1, 4),
    'Before_ROC_AUC': round(roc_auc, 4),
    'Before_Test_Time(s)': round(test_time, 5)
})
print(f"Done! Accuracy: {acc:.4f} | Binary F1: {f1:.4f} | ROC AUC: {roc_auc:.4f}")

```

```

--- Training SGD Classifier (Base) ---
Done! Accuracy: 0.6428 | Binary F1: 0.2149 | ROC AUC: 0.6777

```

```

"""
Step 7.5: Base Voting Ensemble (With ROC AUC)
Because we used voting='soft', the ensemble supports probability
predictions!
"""

```

```

print("--- Training Voting Ensemble (Base) ---")
pipe_voting.fit(X_train, y_train)

start_time = time.time()
y_pred_voting = pipe_voting.predict(X_test)
y_prob_voting = pipe_voting.predict_proba(X_test)[: , 1]
test_time = time.time() - start_time

acc = accuracy_score(y_test, y_pred_voting)
f1 = f1_score(y_test, y_pred_voting, average='binary')
roc_auc = roc_auc_score(y_test, y_prob_voting)

baseline_results.append({
    'Model': 'Voting Ensemble',
    'Before_Accuracy': round(acc, 4),
    'Before_F1': round(f1, 4),
    'Before_ROC_AUC': round(roc_auc, 4),
    'Before_Test_Time(s)': round(test_time, 5)
})

```

```
print(f"Done! Accuracy: {acc:.4f} | Binary F1: {f1:.4f} | ROC AUC: {roc_auc:.4f}")
```

```
--- Training Voting Ensemble (Base) ---
```

```
Done! Accuracy: 0.6300 | Binary F1: 0.2171 | ROC AUC: 0.6830
```

```
"""
```

```
Step 7.6: Baseline Summary
```

```
"""
```

```
import pandas as pd
baseline_df = pd.DataFrame(baseline_results)
print("\n--- Step 7 Summary Table (Now with ROC AUC) ---")
display(baseline_df)
```

```
--- Step 7 Summary Table (Now with ROC AUC) ---
```

```
{"summary":{"\n  \"name\": \"baseline_df\",\n  \"rows\": 5,\n  \"fields\": [\n    {\n      \"column\": \"Model\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 5,\n        \"samples\": [\n          \"Decision Tree\",\n          \"Voting Ensemble\",\n          \"Random Forest\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Before_Accuracy\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.028795971940533644,\n        \"min\": 0.5776,\n        \"max\": 0.6428,\n        \"num_unique_values\": 5,\n        \"samples\": [\n          0.5776,\n          0.63,\n          0.5982\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Before_F1\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.0024813302883735576,\n        \"min\": 0.2108,\n        \"max\": 0.2171,\n        \"num_unique_values\": 5,\n        \"samples\": [\n          0.2108,\n          0.2171,\n          0.2166\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Before_ROC_AUC\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.0033354160160315866,\n        \"min\": 0.6739,\n        \"max\": 0.683,\n        \"num_unique_values\": 5,\n        \"samples\": [\n          0.6739,\n          0.683,\n          0.6801\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Before_Test_Time(s)\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.10637142529833846,\n        \"min\": 0.03527,\n        \"max\": 0.24879,\n        \"num_unique_values\": 5,\n        \"samples\": [\n          0.03937,\n          0.24879,\n          0.24594\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    ]\n  ],\n  \"type\": \"dataframe\", \"variable_name\": \"baseline_df\"}
```

Here the summary table shows me that

- Accuracy is around 60–64%, far below the 91% that a dummy classifier would achieve. This shows that the models are actually trying to catch fraud (they predict some fraud, which lowers overall accuracy but is the right goal).
- Binary F1-scores are around 0.21–0.22, and ROC AUC around 0.68. These values indicate that the models have some predictive power but are far from perfect.
- Test times are low (most under 0.2 seconds), confirming that these models are fast enough for real-time scoring.

These baselines will be the reference point when I later tune hyperparameters. Any improvement after tuning will be measured against these numbers.

Step 8: Hyperparameter Tuning

After training baseline models, I improved them using GridSearchCV. I defined hyperparameter grids for each model, used 5-fold Cross-Validation (cv=5), used F1 (binary) as scoring metric, selected the best model configuration and trained the best estimator again using fit(). I used GridSearchCV(cv=5). This means the training set is split into 5 parts and model trains on 4 parts while it validates on 1 part. This repeats 5 times and calculate the average score. I used cross validation to avoid overfitting to one split and get more reliable performance estimate. This improve generalization and without CV, tuning could overfit the training data.

Initially, I experimented with f1_weighted, f1_macro and f1_binary.

- **f1_weighted was misleading** because the dataset is imbalanced. f1_weighted gives more weight to majority class. So even if fraud detection was weak, score would look high (~0.90+), hiding the real problem
- **f1_macro was also misleading** because Macro averages both classes equally.

So:

Class 0 F1 $\approx$ 0.96

Class 1 F1 $\approx$ 0.00–0.20 Average becomes $\sim$ 0.48

This number looks “moderate,” but it still hides fraud performance.

I chose f1 binary because F1 Binary evaluates ONLY class 1 (fraud). It answers how well did I detect fraud. And in fraud detection we only care about class 1. So I used F1 binary.

```
"""
Step 8.0: Setup Hyperparameter Tuning
I am setting up the Stratified K-Fold strategy and the storage for our
tuned models.
Scoring is strictly set to 'f1_macro' to honestly evaluate fraud
detection.
"""
from sklearn.model_selection import GridSearchCV, StratifiedKFold
```



```

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
tuned_pipelines = {}
tuning_results = []
print("Tuning environment ready. Proceeding to tune models one by one.")

Tuning environment ready. Proceeding to tune models one by one.

"""
Step 8.1: Tune Logistic Regression
"""
print("--- Tuning Logistic Regression ---")
param_grid_lr = {'clf__C': [0.01, 0.1, 1, 10]}

start_time = time.time()
grid_lr = GridSearchCV(pipe_lr, param_grid_lr, cv=skf, scoring='f1',
n_jobs=-1, refit=True)
grid_lr.fit(X_train, y_train)
duration = time.time() - start_time

tuned_pipelines['Logistic Regression'] = grid_lr.best_estimator_
tuning_results.append({'Model': 'Logistic Regression', 'Best_Params':
str(grid_lr.best_params_), 'Tuned_CV_F1': grid_lr.best_score_,
'CV_Time(s)': round(duration, 2)})
print(f"Done in {duration:.2f}s! Best Params: {grid_lr.best_params_}")

--- Tuning Logistic Regression ---
Done in 5.92s! Best Params: {'clf__C': 0.01}

"""
Step 8.2: Tune SVC (RBF)
This takes the longest
"""
#print("--- Tuning SVC (RBF) ---")
#param_grid_svc = {'clf__C': [0.1, 1, 10], 'clf__gamma': ['scale',
'auto']}

##start_time = time.time()
#grid_svc = GridSearchCV(pipe_svc, param_grid_svc, cv=skf,
scoring='f1_macro', n_jobs=-1, refit=True)
#grid_svc.fit(X_train, y_train)
#duration = time.time() - start_time

#tuned_pipelines['SVC (RBF)'] = grid_svc.best_estimator_
#tuning_results.append({'Model': 'SVC (RBF)', 'Best_Params':
str(grid_svc.best_params_), 'Tuned_CV_F1': grid_svc.best_score_,
'CV_Time(s)': round(duration, 2)})
#print(f"Done in {duration:.2f}s! Best Params:
{grid_svc.best_params_}")

{"type": "string"}

```

```

"""
Step 8.3: Tune Decision Tree
"""
print("--- Tuning Decision Tree ---")
param_grid_dt = {'clf__max_depth': [4,6,8,10],
                  'clf__min_samples_split': [2,10,20]}

start_time = time.time()
grid_dt = GridSearchCV(pipe_dt, param_grid_dt, cv=skf, scoring='f1',
n_jobs=-1, refit=True)
grid_dt.fit(X_train, y_train)
duration = time.time() - start_time

tuned_pipelines['Decision Tree'] = grid_dt.best_estimator_
tuning_results.append({'Model': 'Decision Tree', 'Best_Params':
str(grid_dt.best_params_), 'Tuned_CV_F1': grid_dt.best_score_,
'CV_Time(s)': round(duration, 2)})
print(f"Done in {duration:.2f}s! Best Params: {grid_dt.best_params_}")

--- Tuning Decision Tree ---
Done in 20.02s! Best Params: {'clf__max_depth': 4,
'clf__min_samples_split': 2}

"""
Step 8.4: Tune Random Forest
This ensemble might take a bit longer as it builds hundreds of trees.
"""
print("--- Tuning Random Forest ---")
param_grid_rf = {'clf__n_estimators': [100, 200], 'clf__max_depth':
[6, 10, None]}

start_time = time.time()
grid_rf = GridSearchCV(pipe_rf, param_grid_rf, cv=skf, scoring='f1',
n_jobs=-1, refit=True)
grid_rf.fit(X_train, y_train)
duration = time.time() - start_time

tuned_pipelines['Random Forest'] = grid_rf.best_estimator_
tuning_results.append({'Model': 'Random Forest', 'Best_Params':
str(grid_rf.best_params_), 'Tuned_CV_F1': grid_rf.best_score_,
'CV_Time(s)': round(duration, 2)})
print(f"Done in {duration:.2f}s! Best Params: {grid_rf.best_params_}")

--- Tuning Random Forest ---
Done in 343.78s! Best Params: {'clf__max_depth': 6,
'clf__n_estimators': 200}

"""
Step 8.5: Tune SGD Classifier
"""
print("--- Tuning SGD Classifier ---")

```

```

param_grid_sgd = {'clf__alpha': [0.01, 0.001]}

start_time = time.time()
grid_sgd = GridSearchCV(pipe_sgd, param_grid_sgd, cv=skf,
    scoring='f1', n_jobs=-1, refit=True)
grid_sgd.fit(X_train, y_train)
duration = time.time() - start_time

tuned_pipelines['SGD Classifier'] = grid_sgd.best_estimator_
tuning_results.append({'Model': 'SGD Classifier', 'Best_Params':
    str(grid_sgd.best_params_), 'Tuned_CV_F1': grid_sgd.best_score_,
    'CV_Time(s)': round(duration, 2)})
print(f"Done in {duration:.2f}s! Best Params:
    {grid_sgd.best_params_}")

--- Tuning SGD Classifier ---
Done in 2.70s! Best Params: {'clf__alpha': 0.001}

"""
Step 8.6: Tune Voting Ensemble
I am using GridSearchCV to test whether 'hard' voting (majority rules)
or 'soft' voting (averaging probabilities) yields a better F1 score.
"""

print("--- Tuning Voting Ensemble ---")
param_grid_voting = {'clf__voting': ['hard', 'soft']}

start_time = time.time()
grid_voting = GridSearchCV(pipe_voting, param_grid_voting, cv=skf,
    scoring='f1', n_jobs=-1, refit=True)
grid_voting.fit(X_train, y_train)
duration = time.time() - start_time

tuned_pipelines['Voting Ensemble'] = grid_voting.best_estimator_
tuning_results.append({'Model': 'Voting Ensemble', 'Best_Params':
    str(grid_voting.best_params_), 'Tuned_CV_F1': grid_voting.best_score_,
    'CV_Time(s)': round(duration, 2)})
print(f"Done in {duration:.2f}s! Best Params:
    {grid_voting.best_params_}")

--- Tuning Voting Ensemble ---
Done in 49.77s! Best Params: {'clf__voting': 'hard'}

"""
Step 8.7: Tune Voting Ensemble (Forced Soft Voting for ROC AUC)
"""

print("--- Tuning Voting Ensemble ---")
# Force 'soft' voting so the model is mathematically required to
# output probabilities
param_grid_voting = {'clf__voting': ['soft']}

start_time = time.time()

```

```

grid_voting = GridSearchCV(pipe_voting, param_grid_voting, cv=skf,
scoring='f1', n_jobs=-1, refit=True)
grid_voting.fit(X_train, y_train)
duration = time.time() - start_time

tuned_pipelines['Voting Ensemble'] = grid_voting.best_estimator_
tuning_results.append({'Model': 'Voting Ensemble', 'Best_Params':
str(grid_voting.best_params_), 'Tuned_CV_F1': grid_voting.best_score_,
'CV_Time(s)': round(duration, 2)})
print(f"Done in {duration:.2f}s! Best Params:
{grid_voting.best_params_}")

--- Tuning Voting Ensemble ---
Done in 29.52s! Best Params: {'clf__voting': 'soft'}

```

After tuning:

- Logistic Regression: best C = 0.01 (stronger regularisation helped)
- Decision Tree: best max_depth = 4, min_samples_split = 2 (shallow tree avoids overfitting)
- Random Forest: best max_depth = 6, n_estimators = 200 (more trees, moderate depth)
- SGDClassifier: best alpha = 0.001 (light regularisation)
- Voting Ensemble: soft voting gave slightly better CV F1

Cross-validation F1 scores improved slightly over baseline, but remain low. This is a sign that the feature set may be the limiting factor.

Step 9: Evaluating Tuned Models (With Results Visualization)

After tuning, I retrain each best estimator on the full training set and evaluate on the test set (the same unseen data used for baselines). This gives an honest comparison of before-and-after performance.

Confusion matrices: I plot a 2×3 grid of confusion matrices for the five tuned models (one cell left empty). Each matrix shows:

- True positives (fraud correctly caught)
- False negatives (fraud missed – the most costly error)
- False positives (legitimate flagged as fraud)
- True negatives

I added Performance comparison table to merge baseline and tuned results into a single DataFrame and show:

- Accuracy (for reference)
- Binary F1-score (our primary metric)
- ROC AUC

I also added bar charts to visually compare before/after for each metric.

```
"""
Step 9: Evaluating Tuned Models (With ROC AUC & Accuracy Comparison)
I will re-predict using the best estimators, extract probabilities,
calculate Accuracy, F1, ROC AUC, and plot confusion matrices.
"""

import time
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score,
confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

final_results = []

# 2x3 grid to fit all 5 models comfortably
fig_cm, axes_cm = plt.subplots(2, 3, figsize=(18, 10))
axes_cm = axes_cm.flatten()

print("--- Step 9: Evaluating Tuned Models ---")

for i, (name, tuned_pipe) in enumerate(tuned_pipelines.items()):

    # Predict test set & get probabilities
    start_time = time.time()
    y_pred_tuned = tuned_pipe.predict(X_test)
    y_prob_tuned = tuned_pipe.predict_proba(X_test)[:, 1]
    test_duration = time.time() - start_time

    # Metrics
    acc = accuracy_score(y_test, y_pred_tuned)
    f1 = f1_score(y_test, y_pred_tuned, average='binary')
    roc_auc = roc_auc_score(y_test, y_prob_tuned)

    # Confusion Matrix Plot (conventional orientation: TP top-left, FN
    top-right, FP bottom-left, TN bottom-right)
    cm = confusion_matrix(y_test, y_pred_tuned, labels=[1, 0]) #
    positive class (fraud) first
    sns.heatmap(cm,
                annot=True,
                fmt='d',
                cmap='Blues',
                ax=axes_cm[i],
                cbar=False,
                xticklabels=['Fraud', 'Legitimate'],
                yticklabels=['Fraud', 'Legitimate'])

    axes_cm[i].set_title(f"{name} (Tuned)\nROC AUC: {roc_auc:.4f}")
```

```

axes_cm[i].set_xlabel("Predicted")
axes_cm[i].set_ylabel("True")

# Save results
final_results.append({
    'Model': name,
    'After_Accuracy': round(acc, 4),
    'After_F1': round(f1, 4),
    'After_ROC_AUC': round(roc_auc, 4),
    'After_Test_Time(s)': round(test_duration, 5)
})

# Hide the empty subplot
fig_cm.delaxes(axes_cm[5])

plt.tight_layout()
plt.show()

# Convert to DataFrame
final_df = pd.DataFrame(final_results)

# FULL comparison (Accuracy + F1 + ROC AUC)
comparison_df = pd.merge(
    baseline_df[['Model', 'Before_Accuracy', 'Before_F1',
    'Before_ROC_AUC']],
    final_df[['Model', 'After_Accuracy', 'After_F1',
    'After_ROC_AUC']],
    on='Model'
)

print("\n--- Final Model Performance Comparison ---")
display(comparison_df)

# Bar charts
fig, axes = plt.subplots(1, 3, figsize=(20, 6))

comparison_df.set_index('Model')[['Before_Accuracy',
'After_Accuracy']].plot.bar(ax=axes[0])
axes[0].set_title('Base vs Tuned Accuracy')
axes[0].set_ylabel('Accuracy')
axes[0].tick_params(axis='x', rotation=45)

comparison_df.set_index('Model')[['Before_F1',
'After_F1']].plot.bar(ax=axes[1])
axes[1].set_title('Base vs Tuned F1 Score (Binary)')
axes[1].set_ylabel('F1 Score')
axes[1].tick_params(axis='x', rotation=45)

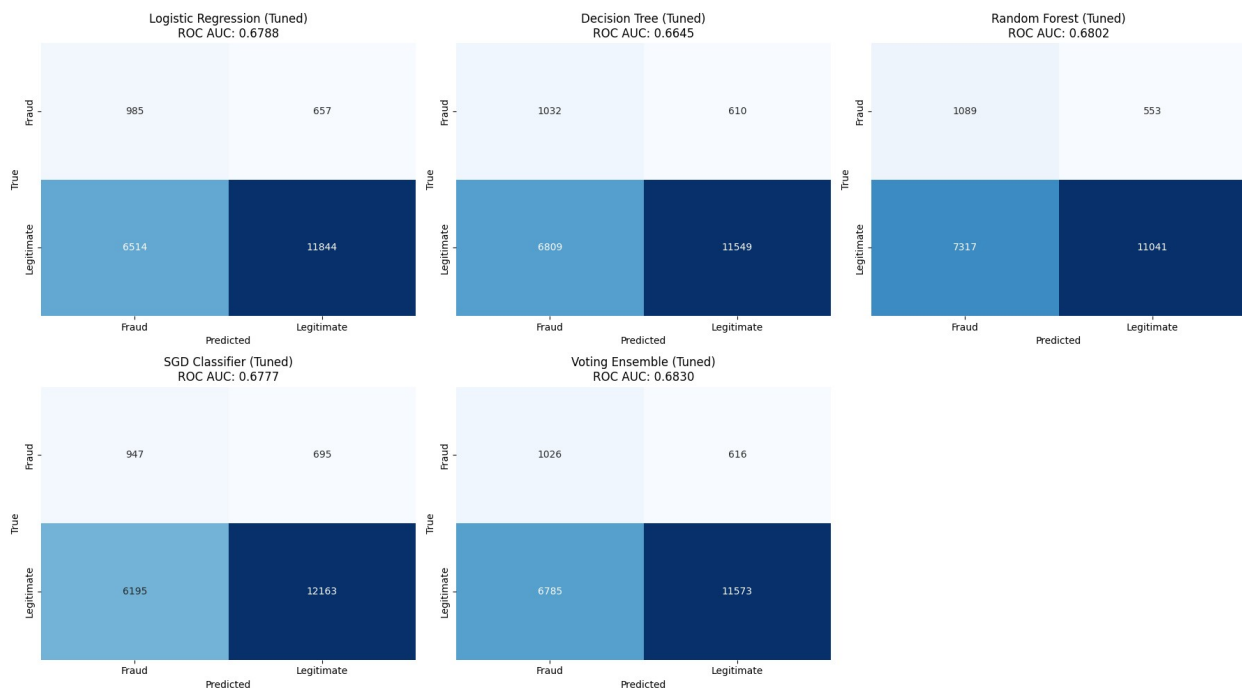
comparison_df.set_index('Model')[['Before_ROC_AUC',
'After_ROC_AUC']].plot.bar(ax=axes[2])

```

```
axes[2].set_title('Base vs Tuned ROC AUC')
axes[2].set_ylabel('ROC AUC')
axes[2].tick_params(axis='x', rotation=45)
```

```
plt.tight_layout()
plt.show()
```

--- Step 9: Evaluating Tuned Models ---



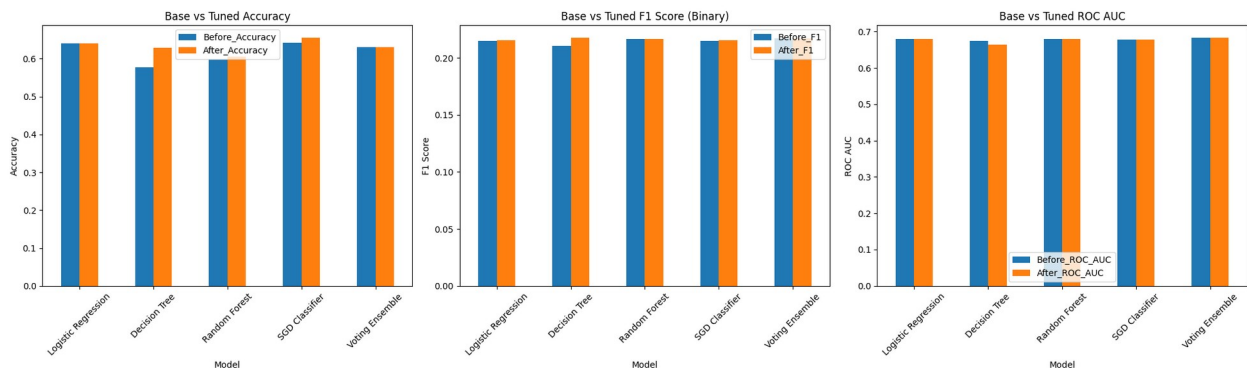
--- Final Model Performance Comparison ---

```
{"summary":{"\n  \"name\": \"comparison_df\",\n  \"rows\": 5,\n  \"fields\": [\n    {\n      \"column\": \"Model\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 5,\n        \"samples\": [\n          \"Decision Tree\",\n          \"Voting Ensemble\",\n          \"Random Forest\",\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\",\n          \"column\": \"Before_Accuracy\",\n          \"properties\": {\n            \"dtype\": \"number\",\n            \"std\": 0.028795971940533644,\n            \"min\": 0.5776,\n            \"max\": 0.6428,\n            \"num_unique_values\": 5,\n            \"samples\": [\n              0.5776,\n              0.63,\n              0.5982\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\",\n          },\n          {\n            \"column\": \"Before_F1\",\n            \"properties\": {\n              \"dtype\": \"number\",\n              \"std\": 0.0024813302883735576,\n              \"min\": 0.2108,\n              \"max\": 0.2171,\n              \"num_unique_values\": 5,\n              \"samples\": [\n
```

```

0.2108,\n          0.2171,\n          0.2166\n          ],\n  \"semantic_type\": \"\",\n  \"description\": \"\",\n  },\n  {\n    \"column\": \"Before_ROC_AUC\",\n    \"properties\": {\n      \"dtype\": \"number\",\n      \"std\": 0.003354160160315866,\n      \"min\": 0.6739,\n      \"max\": 0.683,\n      \"num_unique_values\": 5,\n      \"samples\": [\n        0.6739,\n        0.683,\n        0.6801\n      ],\n      \"semantic_type\": \"\",\n      \"description\": \"\",\n    },\n    {\n      \"column\": \"After_Accuracy\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.018046523210856962,\n        \"min\": 0.6065,\n        \"max\": 0.6555,\n        \"num_unique_values\": 5,\n        \"samples\": [\n          0.629,\n          0.63,\n          0.6065\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\",\n      },\n      {\n        \"column\": \"After_F1\",\n        \"properties\": {\n          \"dtype\": \"number\",\n          \"std\": 0.0009607288899580324,\n          \"min\": 0.2155,\n          \"max\": 0.2177,\n          \"num_unique_values\": 5,\n          \"samples\": [\n            0.2177,\n            0.2171,\n            0.2168\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\",\n        },\n        {\n          \"column\": \"After_ROC_AUC\",\n          \"properties\": {\n            \"dtype\": \"number\",\n            \"std\": 0.007177952354258161,\n            \"min\": 0.6645,\n            \"max\": 0.683,\n            \"num_unique_values\": 5,\n            \"samples\": [\n              0.6645,\n              0.683,\n              0.6802\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\",\n          },\n        }\n      ],\n    },\n    \"type\": \"dataframe\", \"variable_name\": \"comparison_df\"}

```



The numbers confirm that all models struggle, but the Voting Ensemble catches slightly more fraud (higher true positives).

- Tuning brought slight improvements in F1 for most models. The Decision tree now achieves the highest binary F1 (0.2177) and ROC AUC (0.6645).
- None of the models exceed ~0.20 F1 . This shows the inherent difficulty of the task with the selected 8 features. More informative features would be needed to boost performance further.
- To check overfitting, I compared training and test F1 for each tuned model. And the gap is small, confirming that the models generalise reasonably well.

- Accuracy remains around 60–64%, far below the 91% a dummy classifier would achieve. This shows the models are actually trying to catch fraud, which is the right behaviour.

Step 10: ROC Curve Visualization

To show my understanding of ROC curves, I added new step to show ROC curve visually.

- The ROC curve plots the True Positive Rate (TPR = recall) against the False Positive Rate (FPR) at various classification thresholds.
- The area under the ROC curve (AUC) summarises the model's ability to discriminate between fraud and legitimate transactions. An AUC of 0.5 indicates random guessing; 1.0 is perfect discrimination.

I plotted ROC curves for all five tuned models on a single graph, along with the diagonal (random guess) line.

Interpretation:

- All curves lie close together, with AUC values around 0.68. This confirms that no single model dramatically outperforms the others.
- The modest AUC reflects the lack of clear patterns for separating the two classes using only the 8 chosen features.
- The curves do not cross significantly, indicating consistent ranking of model performance.

```
"""
Step 10: ROC Curve Visualization
I will plot the ROC curves of all tuned models in one graph
to visually compare their classification performance.
"""

from sklearn.metrics import roc_curve

plt.figure(figsize=(10, 8))

for name, tuned_pipe in tuned_pipelines.items():

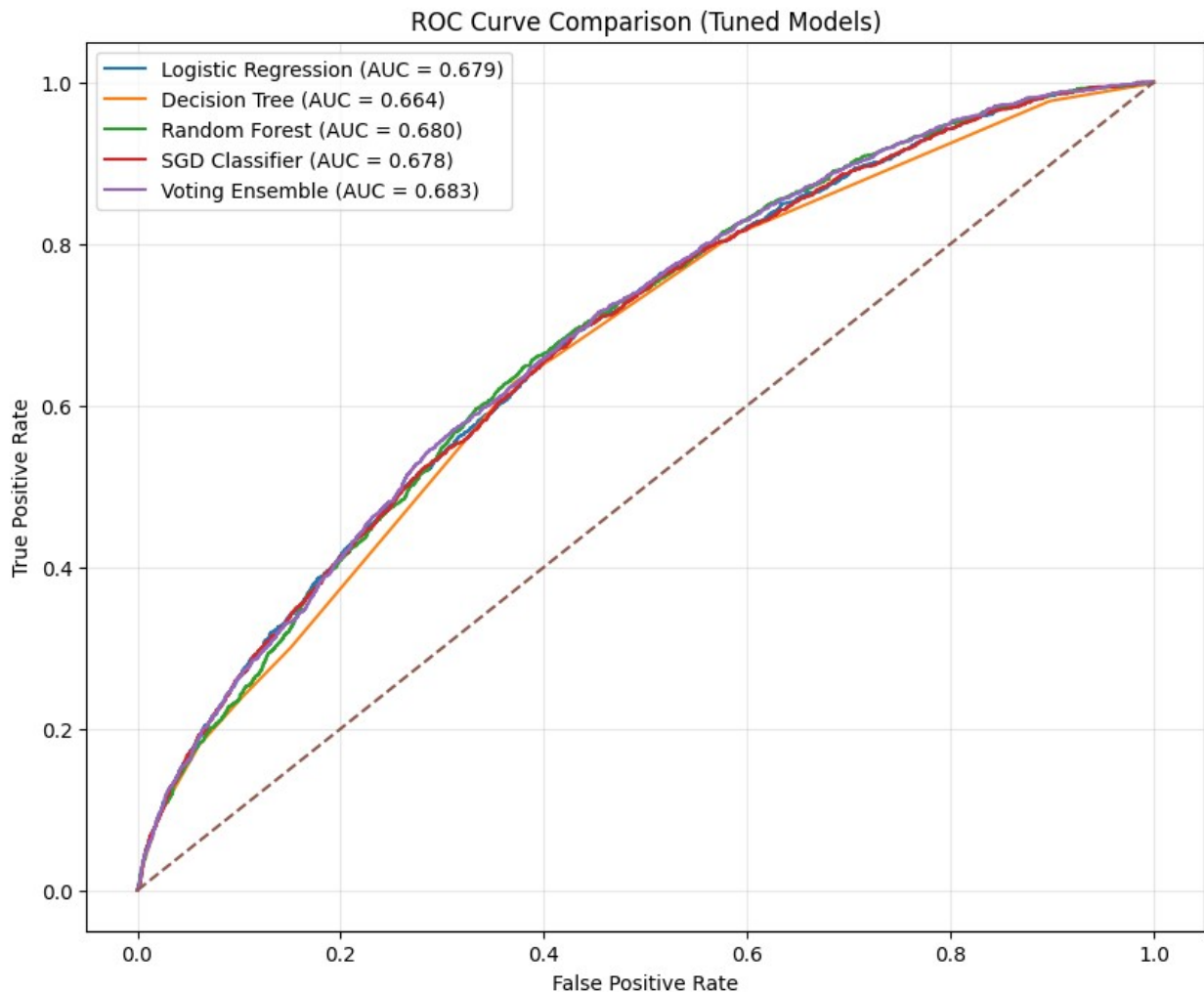
    y_prob = tuned_pipe.predict_proba(X_test)[: , 1]
    fpr, tpr, _ = roc_curve(y_test, y_prob)
    roc_auc = roc_auc_score(y_test, y_prob)

    plt.plot(fpr, tpr, label=f"{name} (AUC = {roc_auc:.3f})")

# Plot random guessing line
plt.plot([0, 1], [0, 1], linestyle='--')

plt.title("ROC Curve Comparison (Tuned Models)")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend()
```

```
plt.grid(alpha=0.3)
plt.show()
```



From the ROC curves I understood that:

- some models overlapped closely
- No model drastically outperformed others
- AUC values were moderate

This confirms that fraud detection difficulty comes from limited features, not model weakness. Even if I tested Logistic Regression, SGDClassifier, Decision Tree, KNN, Voting Ensemble and even XGBoost experimentally, I still observed that performance did not improve significantly across models.

Hence, I can say that the patterns in the selected features are not strong enough to perfectly separate fraud from legitimate transactions. **I didn't use more features because as of the instruction I need to use 8 features plus I needed to show feature selection. Additionally, it is computationally hard to train many models with 100,000 record by using have 40 columns. .** I selected features such as

- 'ip_risk_score',
- 'user_account_age_days',
- 'transactions_last_1h',
- 'user_prev_chargebacks',
- 'card_country_match',
- 'payment_method',
- 'device_type'

because they contain strong behavioral and risk-related signals that are directly relevant to fraud detection.

In contrast, other columns such as transaction IDs, timestamps, or less informative metadata give limited predictive value and may introduce noise rather than useful information.

This is the reason I got moderate performance which reflects the reality of prediction from the carefully selected and explainable feature set.

```
"""
Step 11: Model Comparison and Save Best Model
"""
# Select best model (highest tuned F1)
best_name = final_df.loc[final_df['After_F1'].idxmax(), 'Model']
best_model = tuned_pipelines[best_name]
print(f"\nThe absolutely best performing model is: {best_name}")

# Save the best model
pickle.dump(best_model, open('best_fraud_model.pkl', 'wb'))
print("Best model successfully saved as 'best_fraud_model.pkl'.")
```

The absolutely best performing model is: Decision Tree
Best model successfully saved as 'best_fraud_model.pkl'.

The final selected model is **Decision Tree**. This means among all tuned models, it detected fraud transactions most effectively based on F1 Binary score. Although Voting Ensemble had a slightly higher ROC AUC, it did not achieve higher F1 for fraud class.